МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

Комп’ютерний практикум №4

з курсу «Основи розробки програмного забезпечення на платформі Microsoft.NET»

на тему: «Шаблони проектування. Породжуючі шаблони»

Викладач:
Ліщук К.І.

Виконав:
студент групи ІП-44
Комін Іван

Київ - 2026

Комп‘ютерний практикум № 4. Шаблони проектування. Породжуючі шаблони

Мета: - ознайомитися з основними шаблонами проектування, навчитися застосовувати їх при проектуванні і розробці ПЗ.

Постановка задачі комп‘ютерного практикуму № 4

При виконанні комп‘ютерного практикуму необхідно виконати наступні дії:

1) Вивчити породжуючі патерни. Знати загальну характеристику та призначення кожного з них, особливості реалізації кожного з породжуючих патернів та випадки їх застосування.

2) Реалізувати задачу згідно варіанту, запропонованого нижче у вигляді консольного застосування на мові C#. Розробити інтерфейси та класи з застосування одного або декількох патернів. Повністю реалізувати методи, пов‘язані з реалізацією обраного патерну.

3) Повністю описати архітектуру проекту (призначення методів та класів), особливості реалізації обраного патерну. Для кожного патерну необхідно вказати основні класи та їх призначення,

4) Навести UML-діаграму класів

5) Звіт повинен містити:

a. обґрунтування обраного патерну (чому саме він);

b. опис архітектури проекту (призначення методів та класів);

c. UML-діаграму класів

d. особливості реалізації обраного патерну

e. текст програмного коду

f. скріншоти результатів виконання.

Варіант індивідуальних завдань:

8) Інститут гідрології проєктує роботизовані платформи для дослідження океанів. Кожен робот має відповідати вимогам до глибини занурення, типу даних (температура, тиск, хімічний склад води, біосканування), особливостям регіону (коралові рифи, геотермальні джерела, зони підвищеної сейсмічної активності). Конфігурація включає: корпус із заданою глибиною тиску, сенсорні модулі, автономні блоки живлення, гідролокатори, канали збирання проб, системи навігації. Необхідно реалізувати програмне забезпечення для конструювання роботів, яка дозволяє комбінувати компоненти для створення

спеціалізованих платформ, автоматично перевіряючи сумісність сенсорів, джерел живлення і систем управління, та генеруючи детальну технічну специфікацію.

Опис архітектури проєкту:

1. Обґрунтування обраного патерну

Для реалізації системи конструювання роботизованих платформ обрано породжуючий патерн **Builder (Будівельник)**.

Чому саме цей патерн:

- **Складність об'єкта:** Робот складається з багатьох взаємозалежних компонентів (корпус, сенсори, живлення, навігація), що робить використання звичайних конструкторів неефективним.
- **Покрокове конструювання:** Патерн дозволяє створювати об'єкт поетапно, що є критичним для виконання умови автоматичної перевірки сумісності на кожному кроці або перед завершенням збирання.
- **Різні представлення:** Один і той самий процес будівництва (інтерфейс будівельника) може бути використаний для створення різних типів платформ (глибоководних, прибережних, сейсмічних).
- **Ізоляція логіки:** Логіка перевірки технічних параметрів (наприклад, розрахунок енергобалансу) прихована всередині будівельника і не засмічує код самого класу об'єкта.

2. Опис архітектури проекту

Архітектура побудована на чіткому розподілі відповідальності між класами, що забезпечує гнучкість та розширюваність системи.

Основні класи та їх призначення

- OceanRobot: Зберігає стан готової системи та відповідає за представлення результату користувачеві у вигляді звіту.
- IRobotBuilder: Стандартизує процес створення, дозволяючи за потреби легко замінити одного будівельника іншим без зміни логіки розпорядника.

- OceanRobotBuilder: Відповідає за цілісність об'єкта. Він гарантує, що робот не буде створений, якщо його компоненти технічно несумісні (наприклад, через дефіцит енергії).
- RobotDirector: Спрощує роботу з системою для клієнта, надаючи готові методи для автоматичного збирання типових конфігурацій за один виклик.

3. Призначення методів

Клас OceanRobotBuilder **(Ключові методи)**

- Reset(string name): Ініціалізує процес створення нового порожнього об'єкта робота.
- SetChassis(int depth): Встановлює корпус, автоматично обираючи матеріал (титан або композит) залежно від тиску на заданій глибині.
- SetPowerUnit(string type, double capacity): Підключає джерело живлення з певною потужністю.
- AddSensor(string type, double consumption): Додає спеціалізований модуль у список систем робота.
- Validate(): Внутрішній метод, що перевіряє сумісність: чи вистачає енергії для сенсорів, чи відповідає обладнання регіону, чи є система навігації.
- Build(): Фінальний метод, який запускає валідацію і повертає повністю зібраний та перевірений об'єкт OceanRobot.

Клас RobotDirector

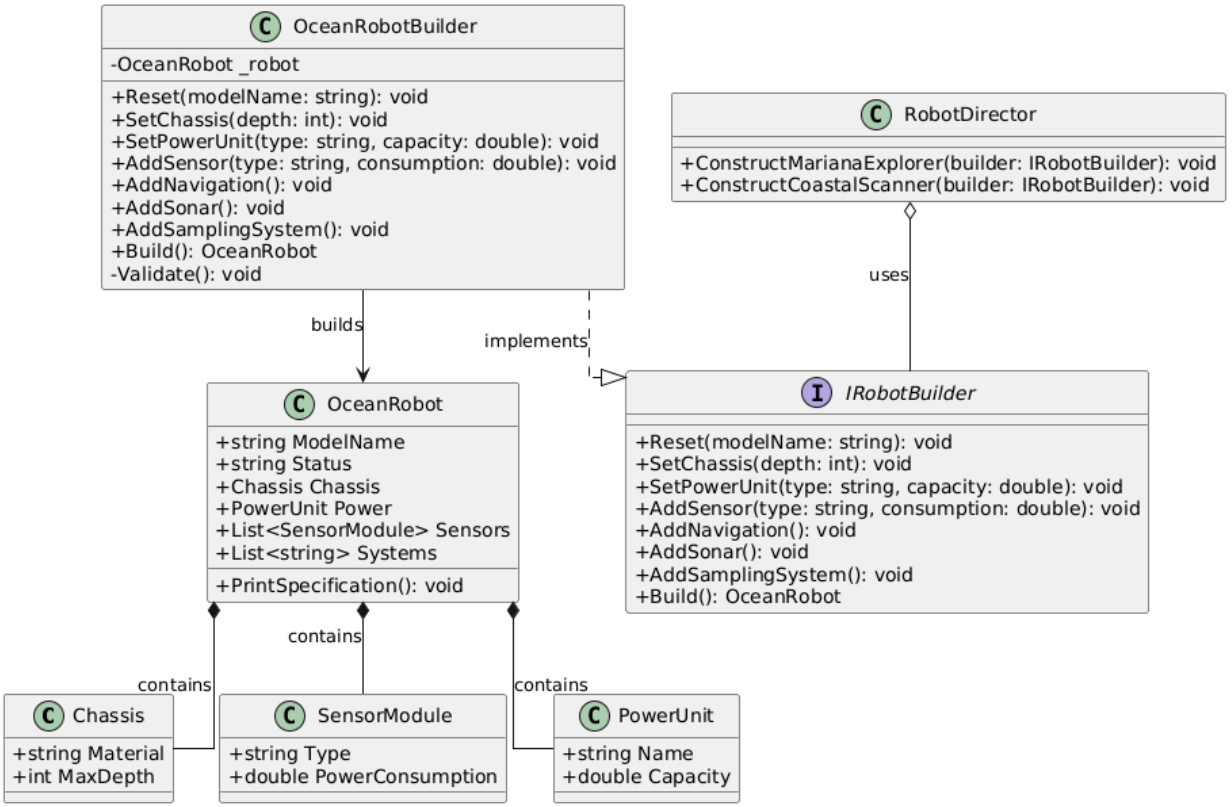
- ConstructMarianaExplorer(): Метод-рецепт, що викликає послідовність команд для створення глибоководного робота з посиленим корпусом.
- ConstructCoastalScanner(): Рецепт для створення легкого маневреного робота для мілководдя.

4. Особливості реалізації патерну

1. **Запобігання помилкам (Safety First):** Клієнтський код не може отримати об'єкт робота, якщо той не пройшов перевірку сумісності. Якщо Validate() виявляє помилку (наприклад, енергоперевантаження), метод Build() викидає виключення (Exception), і об'єкт не створюється.
2. **Гнучкість конфігурації:** Патерн дозволяє легко додавати нові типи сенсорів або систем, не змінюючи інтерфейс Директора або код клієнта.

3. **Автоматизація вибору:** Користувачу достатньо вказати глибину, а Будівельник сам обирає відповідний матеріал корпусу, що зменшує ризик інженерної помилки при проектуванні.
4. **Поділ знань:** Директор знає **що** будувати (місія), а Будівельник знає **як** будувати (технічна реалізація та обмеження).

UML-діаграма класів



Текст програмного коду

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace HydrologyInstitute
{
    public class Chassis
    {
        public string Material { get; set; }
        public int MaxDepth { get; set; }
    }
```

```
    public class SensorModule
    {
        public string Type { get; set; }
        public double PowerConsumption { get; set; }
    }

    public class PowerUnit
    {
        public string Name { get; set; }
        public double Capacity { get; set; }
    }

    public class OceanRobot
    {
        public string ModelName { get; set; }
        public string Status { get; set; } = "In Development";
        public Chassis Chassis { get; set; }
        public PowerUnit Power { get; set; }
        public List<SensorModule> Sensors { get; } = new
List<SensorModule>();
        public List<string> Systems { get; } = new List<string>();

        public void PrintSpecification()
        {

Console.WriteLine("=====================================================");
            Console.WriteLine($"STATUS: {Status}");
            Console.WriteLine($"MODEL:  {ModelName}");

Console.WriteLine("=====================================================");
            Console.WriteLine($"Chassis: {Chassis?.Material} (Rated Depth:
{Chassis?.MaxDepth}m)");
            Console.WriteLine($"Power:   {Power?.Name}
({Power?.Capacity}W)");

            Console.WriteLine("\nSensor Modules:");
            if (!Sensors.Any())
                Console.WriteLine(" [ ] No sensors installed.");
            foreach (var s in Sensors)
                Console.WriteLine($" [+] {s.Type} (Power:
{s.PowerConsumption}W)");

            Console.WriteLine("\nNavigation & Control Systems:");
            foreach (var sys in Systems)
                Console.WriteLine($" [+] {sys}");
```

```
Console.WriteLine("=====================================================\n"
);
        }
    }

    public interface IRobotBuilder
    {
        void Reset(string modelName);
        void SetChassis(int depth);
        void SetPowerUnit(string type, double capacity);
        void AddSensor(string type, double consumption);
        void AddNavigation();
        void AddSonar();
        void AddSamplingSystem();
        OceanRobot Build();
    }

    public class OceanRobotBuilder : IRobotBuilder
    {
        private OceanRobot _robot;

        public void Reset(string modelName)
        {
            _robot = new OceanRobot { ModelName = modelName };
            Console.WriteLine($"[System]: Started designing platform:
{modelName}");
        }

        public void SetChassis(int depth)
        {
            string mat = depth > 5000 ? "Titanium-Alloy" : "Reinforced
Composite";
            _robot.Chassis = new Chassis { MaxDepth = depth, Material = mat
};
            Console.WriteLine($" -> Chassis set for {depth}m depth
({mat}).");
        }

        public void SetPowerUnit(string type, double capacity)
        {
            _robot.Power = new PowerUnit { Name = type, Capacity = capacity
};
```

```
            Console.WriteLine($" -> Power unit connected: {type}
({capacity}W).");
        }

        public void AddSensor(string type, double consumption)
        {
            _robot.Sensors.Add(new SensorModule { Type = type,
PowerConsumption = consumption });
            Console.WriteLine($" -> Sensor module added: {type}.");
        }

        public void AddNavigation() => _robot.Systems.Add("Acoustic
Navigation System");

        public void AddSonar() => _robot.Systems.Add("Side-Scan Sonar");

        public void AddSamplingSystem() => _robot.Systems.Add("Physical
Sampling Channel");

        public OceanRobot Build()
        {
            Validate();
            _robot.Status = "SUCCESSFULLY CONSTRUCTED & VERIFIED";
            Console.WriteLine(
                $"\n[Success]: Platform '{_robot.ModelName}' passed all
compatibility tests."
            );
            return _robot;
        }

        // AUTOMATIC COMPATIBILITY CHECKS
        private void Validate()
        {
            // 1. Mandatory Components
            if (_robot.Chassis == null || _robot.Power == null)
                throw new Exception("Incomplete configuration: Missing
Chassis or Power Unit!");

            // 2. Energy Balance
            double totalPower = _robot.Sensors.Sum(s =>
s.PowerConsumption);
            totalPower += _robot.Systems.Count * 10; // Overhead for
internal systems

            if (totalPower > _robot.Power.Capacity)
```

```
                throw new Exception(
                    $"Energy Overload! Consumption ({totalPower}W) >
Capacity ({_robot.Power.Capacity}W)."
                );

            // 3. Depth-Sensor Compatibility
            if (
                _robot.Chassis.MaxDepth > 7000
                && _robot.Sensors.Any(s => s.Type == "Standard Camera")
            )
                throw new Exception(
                    "Equipment Conflict: Standard cameras cannot withstand
ultra-abyssal depths (>7000m)!"
                );

            // 4. Regional Logic (Example: Seismic Zones)
            if (
                _robot.ModelName.ToLower().Contains("seismic")
                && !_robot.Sensors.Any(s => s.Type.Contains("Pressure"))
            )
                throw new Exception(
                    "Regional Incompatibility: Seismic zone robots must
have a Pressure Sensor for data accuracy!"
                );

            // 5. Dependency Check
            bool hasBioScan = _robot.Sensors.Any(s =>
s.Type.Contains("Bio"));
            bool hasSampling = _robot.Systems.Any(sys =>
sys.Contains("Sampling"));
            if (hasBioScan && !hasSampling)
                Console.WriteLine(
                    "[Warning]: Bio-scanning without a sampling channel
reduces mission efficiency."
                );

            // 6. Safety Minimums
            if (!_robot.Systems.Any(sys => sys.Contains("Navigation")))
                throw new Exception(
                    "Safety Error: Autonomous platforms require a
navigation system to operate!"
                );
        }
    }
```

```
// --- 5. DIRECTOR ---

public class RobotDirector
{
    public void ConstructMarianaExplorer(IRobotBuilder builder)
    {
        builder.Reset("Mariana-X Deep-Seismic");
        builder.SetChassis(11000);
        builder.SetPowerUnit("Radioisotope Generator", 800);
        builder.AddSensor("Pressure Sensor", 50);
        builder.AddSensor("Chemical Composition", 120);
        builder.AddNavigation();
        builder.AddSonar();
    }

    public void ConstructCoastalScanner(IRobotBuilder builder)
    {
        builder.Reset("Coastal-Scan v2");
        builder.SetChassis(200);
        builder.SetPowerUnit("Li-ion Battery Pack", 150);
        builder.AddSensor("Bio-Scanner", 40);
        builder.AddNavigation();
        builder.AddSamplingSystem();
    }
}

// --- 6. CLIENT CODE ---

class Program
{
    static void Main()
    {
        var builder = new OceanRobotBuilder();
        var director = new RobotDirector();

        // Platform 1: Mariana Explorer (Valid)
        try
        {
            director.ConstructMarianaExplorer(builder);
            OceanRobot deepRobot = builder.Build();
            deepRobot.PrintSpecification();
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[FATAL ERROR]: {ex.Message}");
```

```
            }

            // Platform 2: (Invalid) Overpowered Custom Build
            try
            {
                Console.WriteLine("Attempting to build an overpowered
custom platform...");
                builder.Reset("Experimental-Fail-Unit");
                builder.SetChassis(1000);
                builder.SetPowerUnit("Small Battery", 20);
                builder.AddSensor("Heavy LiDAR", 150);
                builder.AddNavigation();

                builder.Build();
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[CONSTRUCTION REJECTED]:
{ex.Message}");
                Console.ResetColor();
            }

            // Platform 3: (Invalid) Seismic Robot without Pressure Sensor
            try
            {
                Console.WriteLine(
                    "\nAttempting to build a Seismic robot without Pressure
sensors..."
                );
                builder.Reset("Seismic-Seeker-Alpha");
                builder.SetChassis(5000);
                builder.SetPowerUnit("Fuel Cell", 500);
                builder.AddSensor("Bio-Scanner", 30);
                builder.AddNavigation();

                builder.Build();
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[CONSTRUCTION REJECTED]:
{ex.Message}");
                Console.ResetColor();
            }
```

```
        }
    }
}
```

Скріншоти результатів виконання

```
[System]: Started designing platform: Mariana-X Deep-Seismic
 -> Chassis set for 11000m depth (Titanium-Alloy).
 -> Power unit connected: Radioisotope Generator (800W).
 -> Sensor module added: Pressure Sensor.
 -> Sensor module added: Chemical Composition.

[Success]: Platform 'Mariana-X Deep-Seismic' passed all compatibility tests.
=====================================================
STATUS: SUCCESSFULLY CONSTRUCTED & VERIFIED
MODEL:  Mariana-X Deep-Seismic
=====================================================
Chassis: Titanium-Alloy (Rated Depth: 11000m)
Power:   Radioisotope Generator (800W)

Sensor Modules:
 [+] Pressure Sensor (Power: 50W)
 [+] Chemical Composition (Power: 120W)

Navigation & Control Systems:
 [+] Acoustic Navigation System
 [+] Side-Scan Sonar
=====================================================
```

```
Attempting to build an overpowered custom platform...
[System]: Started designing platform: Experimental-Fail-Unit
 -> Chassis set for 1000m depth (Reinforced Composite).
 -> Power unit connected: Small Battery (20W).
 -> Sensor module added: Heavy LiDAR.
[CONSTRUCTION REJECTED]: Energy Overload! Consumption (160W) > Capacity (20W).
```

```
Attempting to build a Seismic robot without Pressure sensors...
[System]: Started designing platform: Seismic-Seeker-Alpha
 -> Chassis set for 5000m depth (Reinforced Composite).
 -> Power unit connected: Fuel Cell (500W).
 -> Sensor module added: Bio-Scanner.
[CONSTRUCTION REJECTED]: Regional Incompatibility: Seismic zone robots must have a Pressure Sensor for data accuracy!
```

Висновок: Під час виконання комп'ютерного практикуму було досліджено та практично реалізовано породжуючий патерн проектування **Builder** для розробки системи конструювання роботизованих платформ дослідження океанів. Застосування даного шаблону дозволило ефективно відокремити покроковий процес збирання складного об’єкта від його технічного

представлення, що забезпечило високу гнучкість при створенні різних конфігурацій роботів залежно від глибини занурення та специфічних вимог регіону дослідження. Ключовою перевагою обраної архітектури стала інтеграція механізму автоматичної перевірки сумісності компонентів (енергетичного балансу, стійкості до тиску та логічної відповідності модулів), що гарантує технічну цілісність платформи та запобігає виникненню критичних інженерних помилок ще на етапі проектування. Таким чином, використання патернів проектування дозволило створити масштабовану та надійну програмну систему, яка легко адаптується до нових типів обладнання та умов експлуатації